TP MI-GUIDE

INSTALLATION ET ARCHITECTURE

DE BASE DE DJANGO

Introduction

Dans ce TP, nous allons découvrir les bases techniques nécessaires pour commencer à développer une application web avec Django.

Django est un framework web open-source écrit en Python qui permet de développer des applications web robustes, sécurisées et évolutives. De nombreuses plateformes connues utilisent Django, notamment :

- Instagram (dans ses premières versions)
- Pinterest
- Mozilla

Le framework Django permet d'accélérer le développement en fournissant une architecture prête à l'emploi et un ensemble d'outils permettant de gérer :

- les URL
- les bases de données
- les templates HTML
- l'authentification
- la sécurité
- l'administration du site

Avant de pouvoir développer une application Django, il est nécessaire de préparer un environnement de développement complet. Ce TP a donc pour objectif de vous familiariser avec :

- l'installation de Python
- l'installation de Django
- l'utilisation de l'invite de commandes

- la création d'un environnement virtuel
- la création d'un projet Django
- la compréhension de l'architecture générée automatiquement par Django

Objectifs pédagogiques

À la fin de ce TP, vous devez être capable de :

- Installer Django avec pip
- Comprendre le rôle des environnements virtuels
- Créer un projet Django
- Comprendre la structure d'un projet Django
- Exécuter un serveur Django local
- Identifier le rôle des fichiers générés automatiquement

Installation de Python

Vérifier si Python est déjà installé

Avant d'installer Python, il est important de vérifier s'il est déjà présent sur votre machine. Ouvrez un terminal ou une invite de commande et exécutez :

```
python --version
```

ou

```
python3 --version
```

Si Python est installé, vous devriez obtenir un résultat similaire à :

```
Python 3.11.4
```

Si la commande n'est pas reconnue, cela signifie que Python n'est pas installé.

Télécharger Python

Télécharger depuis le site officiel : https://www.python.org

Dans la section Downloads, téléchargez la dernière version stable de Python.

Lors de l'installation :

Assurez-vous de cocher :

Add Python to PATH

Cette option permet d'utiliser Python depuis n'importe quel terminal.

Vérification après installation

Après installation, relancez le terminal puis tapez :

```
python --version
```

Le système doit maintenant afficher la version installée.

Création d'un environnement virtuel

Pourquoi utiliser un environnement virtuel ?

Dans un projet Python, il est courant d'utiliser plusieurs bibliothèques. Cependant, différents projets peuvent nécessiter des versions différentes d'une même bibliothèque.

Sans environnement virtuel, ces versions peuvent entrer en conflit. Un environnement virtuel permet :

- d'isoler les dépendances
- d'éviter les conflits
- de reproduire un environnement de développement

Installation

Ouvrez un terminal (ou invite de commandes) et créez un nouveau dossier pour votre projet, puis naviguez à l'intérieur.

Créez un environnement virtuel. Nous l'appellerons monenv.

```
python -m venv monenv
```

Activez l'environnement virtuel. La commande varie selon votre système d'exploitation :

```
monenv\Scripts\activate
```

Introduction au gestionnaire de paquets pip

Python utilise un gestionnaire de paquets appelé pip. Il permet d'installer des bibliothèques Python depuis Internet.

Exemple :

```
pip install nom_du_package
```

pip télécharge alors automatiquement la bibliothèque et ses dépendances.

Installation de Django

Maintenant que Python est installé, nous pouvons installer Django. Dans le terminal, exécutez :

```
pip install django
```

Le système va télécharger Django et ses dépendances

Le résultat doit ressembler à : Successfully installed django

Vérification de l'installation

Tapez :

```
django-admin --version
```

Résultat attendu :

```
4.x.x
```

Cela confirme que Django est correctement installé.

Création d'un projet Django

Une fois Django installé, nous pouvons créer un projet.

Dans Django, un projet est un site web complet, contenant la configuration globale. Une application est un module qui remplit une fonction spécifique (ex: un blog, un système d'utilisateurs). Un projet peut contenir plusieurs application.

Initialisation du projet et de l'application

Créez un nouveau projet Django que nous nommerons myproject.

```
django-admin startproject myproject
```

Naviguez dans le dossier du projet qui vient d'être créé.

```
cd myproject
```

```
mon_projet_django/
├── manage.py
└── mon_projet_django/
    ├── __init__.py
    ├── asgi.py
    ├── settings.py
    ├── urls.py
    └── wsgi.py
```

Comprendre la structure du projet

manage.py

Ce fichier permet d'exécuter les commandes Django. Exemples :

python manage.py runserver

python manage.py migrate

python manage.py startapp

settings.py

C'est le fichier de configuration principal du projet. Il contient :

- les applications installées
- les bases de données
- les templates
- les fichiers statiques

Une mauvaise configuration peut empêcher le projet de fonctionner.

urls.py

Ce fichier définit les routes URL du projet. Exemple :

site.com/

site.com/blog/

site.com/contact/

Chaque URL est associée à une fonction Python.

wsgi.py

Permet l'interaction entre Django et le serveur web (expliqué en détails sur le support du cours).

asgi.py

Version moderne utilisée pour les applications asynchrones (expliqué en détails sur le support du cours).

Création d'une application dans notre projet Django

Créez une application pour notre projet. Appelons-la myapp.

```
python manage.py startapp myapp
```

Exécution du serveur Django

Accédez au dossier contenant manage.py

Puis exécutez :

```
python manage.py runserver
```

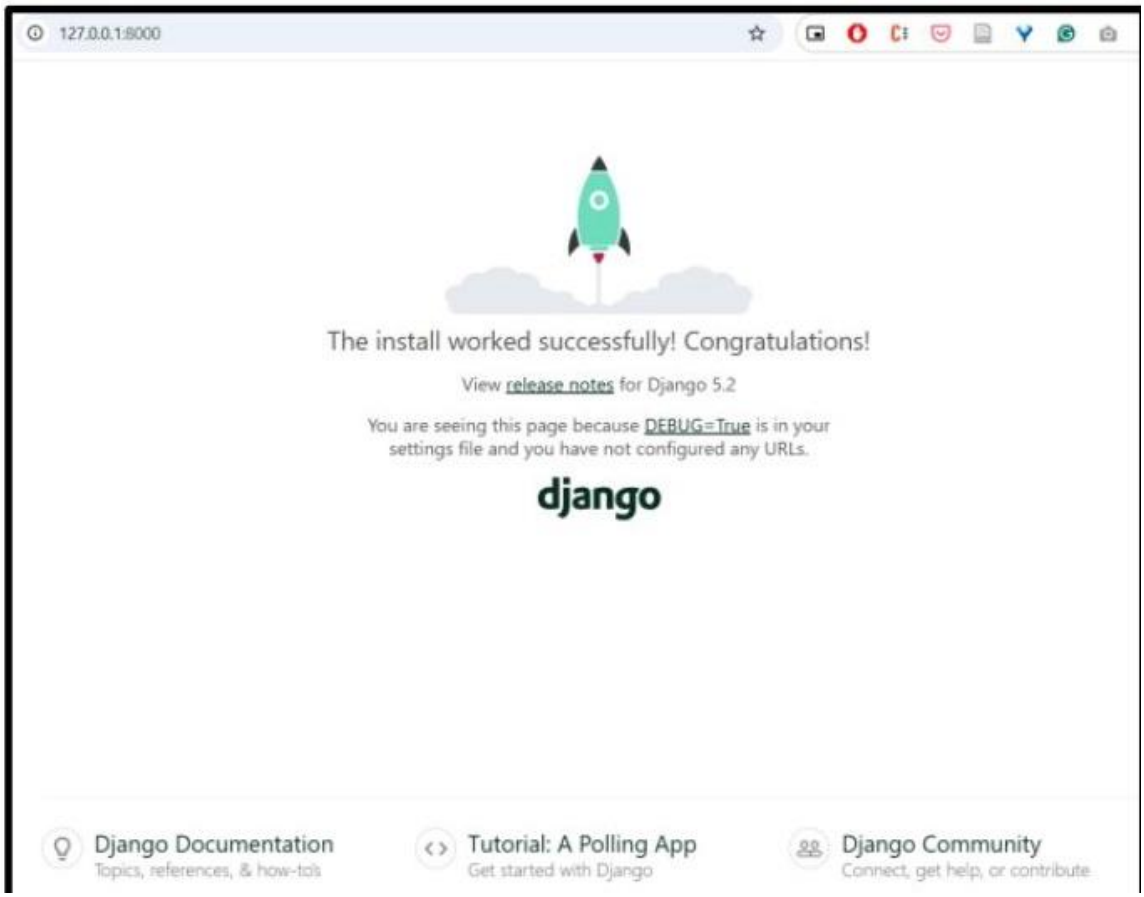
Résultat :

```
Starting development server at http://127.0.0.1:8000/
```

Ouvrez votre navigateur :

```
http://127.0.0.1:8000
```

La page d'accueil Django doit apparaître.



Vues, URLs et Templates : Afficher la Première Page

Nous allons maintenant créer notre première page web. Le flux dans Django suit l'architecture MVT (Modèle-Vue-Template) :

Une URL demandée par l'utilisateur est interceptée.

Elle est dirigée vers une Vue (une fonction Python).

La Vue exécute la logique, interagit avec les Modèles (base de données) si nécessaire, et renvoie une réponse, souvent en utilisant un Template (fichier HTML).

Configuration des URLs et des Vues

Créez un fichier urls.py dans votre application myapp. Ce fichier gérera les URLs spécifiques à cette application.

Vérifier la création automatique d'une vue simple dans myapp/views.py. Pour l'instant, elle ne fera rien de spécial.

Ajoutez un chemin d'URL dans myapp/urls.py pour lier l'URL racine de l'application à notre vue index.

```
›2 > myproject > myapp > urls.py > .
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name="index")
]
```

Incluez les URLs de l'application dans le projet principal. Modifiez myproject/urls.py pour qu'il redirige les requêtes vers myapp.urls.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myapp.urls'))
]
```

Utilisation des Templates

Exemple de fonction sur views.py

```
from django.shortcuts import render
from django.http import HttpResponse

# Create your views here.
def index(request):
    return HttpResponse('<h1> welcome </h1>')
```

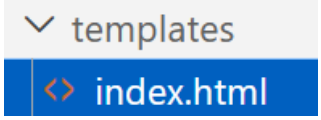
Écrire du HTML directement dans les vues n'est pas une bonne pratique. Utilisons le moteur de templates de Django.

Créez un dossier templates à la racine de votre projet (au même niveau que manage.py).

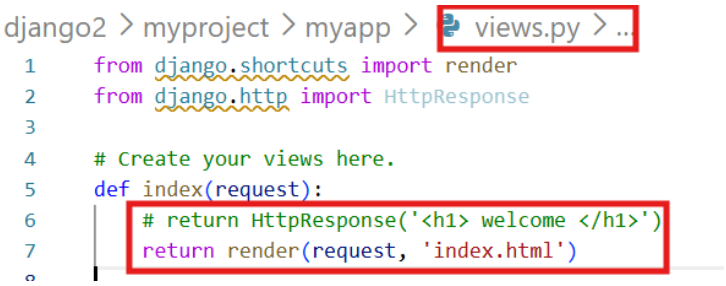
Configurez Django pour qu'il trouve ce dossier. Dans myproject/settings.py, trouvez la variable TEMPLATES et modifiez la clé 'DIRS'.

```
django2 > myproject > myproject > settings.py > ...
51
52  ROOT_URLCONF = 'myproject.urls'
53
54  TEMPLATES = [
55      {
56          'BACKEND': 'django.template.backends.django.DjangoTemplates',
57          'DIRS': [BASE_DIR, 'templates'],
58          'APP_DIRS': True,
59          'OPTIONS': {
60              'context_processors': [
61                  'django.template.cont   (module) context_processors
62                  'django.contrib.auth.context_processors.auth',
63                  'django.contrib.messages.context_processors.messages',
```

Créez votre premier template. Dans le dossier templates, créez un fichier index.html.



Modifiez la vue pour utiliser le template. Dans myapp/views.py, utilisez la fonction render.



Passer des données dynamiques au Template

La vraie puissance des templates réside dans leur capacité à afficher des données dynamiques.

```
# Create your views here.
def index(request):
    # return HttpResponse('<h1> welcome </h1>')
    name = 'Sara'
    return render(request, 'index.html', {'name': name})
```

Pour ce faire, on peut aussi passer un dictionnaire de "contexte" à la fonction render.

Modifiez myapp/views.py pour passer un contexte.

```
# Create your views here.
def index(request):
    # return HttpResponse('<h1> welcome </h1>')
    name = 'Sara'
    context = {
        'name': 'Sara',
        'age': 30,
        'morrocan': True
    }
    return render(request, 'index.html', context)
```

Affichez ces données dans templates/index.html en utilisant la syntaxe à double accolade {{ }}.

Gestion des Formulaires : Création d'un Compteur de Mots

Les formulaires sont essentiels pour interagir avec les utilisateurs. Nous allons créer un formulaire simple qui compte le nombre de mots dans un texte soumis.

Ajoutez un formulaire HTML dans templates/index.html.

```
<h1>Input text below</h1>
<form method="">
    <textarea name="text" rows="25" cols="25" id=""></textarea> <br>
    <input type="submit" value="submit">
</form>
```

Créez la vue et l'URL pour le compteur.

```
django2 > myproject > myapp > views.py > index

def counter(request):
    return render(request, 'counter.html')
```

```
django2 > myproject > myapp > urls.py > ...

from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name="index"),
    path('counter', views.counter, name="counter")
]
```

Créez le template de résultat counter.html dans votre dossier templates.

```
<h1>Input text below</h1>
<form method="" action="counter">
    <textarea name="text" rows="25" cols="25" id=""></textarea> <br>
    <input type="submit" value="submit">
</form>
```

```
def counter(request):
    text = request.GET['text']
    amount_of_words = len(text.split())
    return render(request, 'counter.html', {'amount': amount_of_words})
```

Testez votre compteur ! Allez sur la page d'accueil, entrez du texte et cliquez sur le bouton.

Tester le formulaire POST

```
<body>
    <h1>Input text below</h1>
    <form method="POST" action="counter">
        <textarea name="text" rows="25" cols="25" id=""></textarea> <br>
        <input type="submit" value="submit">
    </form>
</body>
</html>
```

Forbidden (403)

CSRF verification failed. Request aborted.

You are seeing this message because this site requires a CSRF cookie when submitting forms. This c

If you have configured your browser to disable cookies, please re-enable them, at least for this site, o

```
<form method="POST" action="counter">
    {% csrf_token %}
    <textarea name="text" rows="25" cols="25"
    <input type="submit" value="submit">
</form>
```

MultiValueDictKeyError at /counter

'text'

Request Method:	POST
Request URL:	http://127.0.0.1:8000/counter
Django Version:	5.2.12
Exception Type:	MultiValueDictKeyError
Exception Value:	'text'
Exception Location:	C:\wamp64\www\fstt_prof\django\django2\mor
Raised during:	myapp.views.counter

```
def counter(request):
    text = request.POST['text']
    amount_of_words = len(text.split())
    return render(request, 'counter.html', {'amount': amount_of_words})
```

Intégration des Fichiers Statiques (CSS/JS du template Bootstrap)

Les fichiers statiques sont les fichiers CSS, JavaScript, images, etc., qui ne changent pas. Nous allons intégrer un template Bootstrap pour donner un look professionnel à notre site.

Téléchargez un template. Pour ce TP, nous utiliserons le template gratuit "OnePage" de BootstrapMade. Téléchargez-le et décompressez l'archive.

Organisez les fichiers statiques.

- À la racine de votre projet Django (au même niveau que manage.py), créez un dossier nommé static.
- Copiez les dossiers assets (ou css, js, img, etc.) du template téléchargé dans ce dossier static.

SETTINGS.PY

Configurez Django pour les fichiers statiques. Dans myproject/settings.py, ajoutez la configuration suivante à la fin du fichier :

```
import os

STATIC_URL = 'static/'
STATICFILES_DIRS = (os.path.join(BASE_DIR, 'static'),)
```

Add {% load static %}

```
<meta name="viewport" content="width=device-width, initial-scale=
<link rel="stylesheet" href="{% static 'style.css' %}">
<title>Document</title>
```

TemplateSyntaxError at /

Invalid block tag on line 7: 'static'. Did you forget to register or load this tag?

Request Method:	GET
Request URL:	http://127.0.0.1:8000/?text=dfdf
Django Version:	5.2.12
Exception Type:	TemplateSyntaxError
Exception Value:	Invalid block tag on line 7: 'static'. Did you forget to register or load this tag?

Inclure le template dans le projet

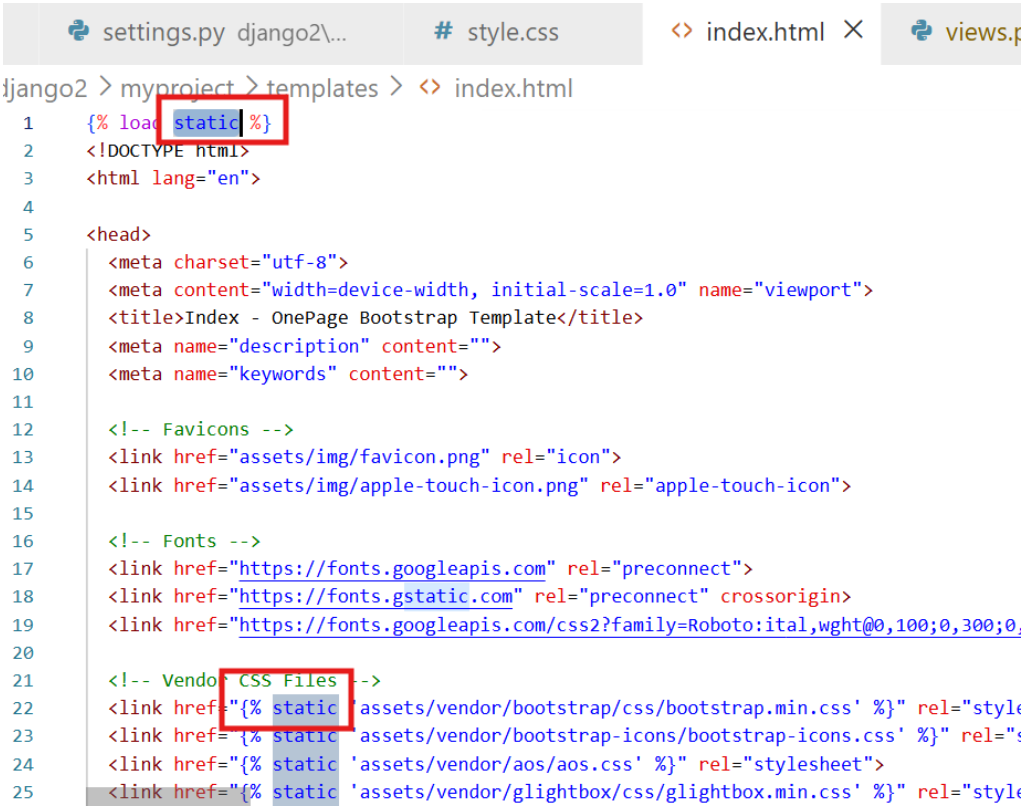
Copier tout le dossier assets dans le dossier static.

Ouvrez le fichier index.html du template que vous avez téléchargé sur ONEPAGE et copiez tout son contenu.

Collez ce contenu dans votre fichier templates/index.html, en remplaçant tout ce qui s'y trouve.

Eu haut de ce fichier, juste après la balise <!DOCTYPE html>, ajoutez la balise {% load static %}. C'est indispensable pour que Django puisse générer les URLs des fichiers statiques.

Recherchez tous les liens vers des fichiers CSS et JS (ex: <link href="assets/css/style.css" ...>) et les images (<img src="assets/img/logo.png" ...>). Modifiez-les pour utiliser la balise {% static %}.



Modèles de Données et Migrations

Les modèles sont la représentation de vos données en classes Python. L'ORM (Object-Relational Mapper) de Django se charge de traduire ces classes en tables de base de données.

Définissez un modèle dans myapp/models.py. Créons un modèle simple pour représenter des "fonctionnalités" de notre site.

Enregistrez votre application. Pour que Django prenne en compte les modèles de votre application, elle doit être déclarée dans INSTALLED_APPS dans myproject/settings.py.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp'
]
```

Créez et appliquez les migrations. Les migrations sont des fichiers qui décrivent les changements à apporter à la base de données.

Un modèle simple qui ne sera pas intégré sur les migrations :

```
django2 > myproject > myapp > models.py > ...
1   from django.db import models
2
3   # Create your models here.
4   class Feature:
5       id: int
6       name: str
7       details: str
```

Un modèle qui peut être migré :

```
django2 > myproject > myapp > models.py > Feature > details
1   from django.db import models
2
3   # Create your models here.
4   class Feature(models.Model):
5       # id: int  every model has an id when it s created
6       name = models.CharField(max_length=100)
7       details = models.CharField(max_length=100)
```

L'Interface d'Administration de Django

Django fournit une interface d';administration prête à l'emploi pour gérer les données de vos modèles. C'est l'une de ses fonctionnalités les plus puissantes.

Créez un super-utilisateur pour pouvoir vous connecter à l'interface d'administration.

```
django2 > myproject > myapp > admin.py
1   from django.contrib import admin
2   from .models import Feature
3
4   # Register your models here.
5   admin.site.register(Feature)
```

Enregistrez votre modèle dans l'admin. Pour que le modèle Feature apparaisse dans l'interface, vous devez l'enregistrer dans myapp/admin.py.

```
django2 > myproject > myapp > admin.py
1   from django.contrib import admin
2   from .models import Feature
3
4   # Register your models here.
5   admin.site.register(Feature)
```

Explorez l'interface d'administration.

- Lancez le serveur : python manage.py runserver.
- Allez à l'adresse http://127.0.0.1:8000/admin/.
- Connectez-vous avec les identifiants du super-utilisateur que vous venez de créer.
- Vous devriez voir une section "MYAPP" avec un lien "Features". Cliquez dessus et ajoutez quelques fonctionnalités via le bouton "Add feature".

Affichez les données de la base de données dans votre template. Modifions la vue index pour récupérer les fonctionnalités et les afficher.

```python
# Create your views here.
def index(request):
    features = Feature.objects.all()
    return render(request, 'index.html', {'features': features})
```

Authentification des Utilisateurs

Django inclut un système d'authentification complet. Nous allons implémenter l'inscription, la connexion et la déconnexion.

Inscription (Register)

Créez le template register.html dans votre dossier templates.

```html
<body>
    <h1>test register</h1>
    <form action="register" method="post">
        <p>Username</p>
        <input type="text" name="username" id="username"> <br>
        <p>Email</p>
        <input type="email" name="email" id="email"> <br>
        <p>Password</p>
        <input type="password" name="password" id="password"> <br>
        <p>Password 2</p>
        <input type="password1" name="password1" id="password1"> <br>

        <input type="submit" value="SUBMIT">
    </form>
</body>
```

Ajoutez l'URL et la vue.

```
django2 > myproject > myapp > urls.py > ...
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name="index"),
    path('counter', views.counter, name="counter"),
    path('register', views.register, name="register")
]
```

Configurer la fonction register dans views.py

Imports

```
from django.shortcuts import render, redirect
from django.http import HttpResponse
from django.contrib.auth.models import User, auth
from django.contrib import messages
from .models import Feature
```

```python
def register(request):
    if request.method == 'POST':
        username = request.POST['username']
        email = request.POST['email']
        password = request.POST['password']
        password2 = request.POST['password2']

        if password == password2:
            if User.objects.filter(email=email).exists():
                messages.info(request, 'email existe deja')
                return redirect('register')
            elif User.objects.filter(username=username).exists():
                messages.info(request, 'username existe deja')
                return redirect('register')
            else:
                user = User.objects.create_user(username=username, email=email, password=password)
                user.save()
                return redirect('login')
        else:
            messages.info(request, 'MDP non identiques')
            return redirect('register')
    else:
        return render(request, 'register.html')
```

Connexion (Login)

Créez le template login.html.

```python
def login(request):
    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']

        user = auth.authenticate(username=username, password=password)
        if user is not None:
            auth.login(request, user)
            return redirect('/')
        else:
            messages.info(request, 'Cred invalid')
    else:
        return render(request, 'login.html')
```

Ajoutez l'URL et la vue.

```python
    path('register', views.register, name="register"),
    path('login', views.login, name="login"),
    path('logout', views.logout, name="logout")
]
```

```
{% if user.is_authenticated %}
<a class="btn-getstarted" href="logout">Logout</a>
{% else %}
<a class="btn-getstarted" href="login">Login</a>
{% endif %}
```

Déconnexion (Logout)

La déconnexion est plus simple car elle ne nécessite pas de formulaire.

```
def logout(request):
    auth.logout(request)
    return redirect('index')
```

Ajoutez l'URL et la vue.

```
    path('register', views.register, name="register"),
    path('login', views.login, name="login"),
    path('logout', views.logout, name="logout")
]
```

Travail demandé

Chaque étudiant doit remettre un rapport Word contenant :

- les commandes exécutées sur le terminal avec chacun le nom de son ordi
- les captures d'écran
- les explications des concepts

Un dépôt GitHub PUBLIC contenant :

- le projet Django